C Lab Programs - Data Structures (All 15 Programs)

1. Largest and Smallest Element in an Array

```
#include <stdio.h>
int main() {
    int n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    int max = arr[0], min = arr[0];
    for(i = 1; i < n; i++) {
        if(arr[i] > max) max = arr[i];
        if(arr[i] < min) min = arr[i];
    }
    printf("Largest element: %d\n", max);
    printf("Smallest element: %d\n", min);
    return 0;
}
```

```
Sample Input:
5
10 25 3 45 8
Sample Output:
Largest element: 45
Smallest element: 3
```

2. Factorial Using Recursion

```
#include <stdio.h>
int factorial(int n) {
    if(n==0 || n==1) return 1;
    return n * factorial(n-1);
}
int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    printf("Factorial of %d is %d\n", n, factorial(n));
    return 0;
}
```

```
Sample Input:
5
Sample Output:
Factorial of 5 is 120
```

3. Fibonacci Using Recursion

```
#include <stdio.h>
int fibonacci(int n) {
    if(n<=1) return n;
    return fibonacci(n-1) + fibonacci(n-2);
}
int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Fibonacci number at position %d is %d\n", n, fibonacci(n));
    return 0;
}
```

```
Sample Input:
7
Sample Output:
Fibonacci number at position 7 is 13
```

4. Singly Linked List

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node* next; };
struct Node* head = NULL;

void insert_at_beginning(int data){
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data; newNode->next = head; head = newNode;
}
void insert_at_end(int data){
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data; newNode->next = NULL;
    if(head==NULL){ head=newNode; return; }
    struct Node* temp=head;
    while(temp->next!=NULL) temp=temp->next;
    temp->next=newNode;
}
void delete_node(int data){
    struct Node *temp=head,*prev=NULL;
    while(temp && temp->data!=data){ prev=temp; temp=temp->next; }
    if(!temp) return;
    if(prev) prev->next=temp->next; else head=temp->next;
    free(temp);
}
void traverse(){
    struct Node* temp=head;
    while(temp){ printf("%d -> ",temp->data); temp=temp->next; }
    printf("NULL\n");
}
int main(){
    insert_at_beginning(10);
    insert_at_end(20);
    insert_at_end(30);
    traverse();
    delete_node(20);
    traverse();
    return 0;
}
```

```
Sample Output:
10 -> 20 -> 30 -> NULL
10 -> 30 -> NULL
```

5. Stack Using Linked List

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node* next; };
struct Node* top=NULL;

void push(int data){
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=data; newNode->next=top; top=newNode;
}
int pop(){
    if(!top){ printf("Stack Underflow\n"); return -1; }
    struct Node* temp=top; int val=temp->data;
    top=top->next; free(temp); return val;
}
int peek(){ return top ? top->data : -1; }
int empty(){ return top==NULL; }

int main(){
    push(10); push(20);
    printf("Top: %d\n", peek());
    printf("Popped: %d\n", pop());
    return 0;
}
```

```
Sample Output:
Top: 20
Popped: 20
```

6. Queue Using Linked List

```
#include <stdio.h>
#include <stdlib.h>
struct Node { int data; struct Node* next; };
struct Node *front=NULL,*rear=NULL;

void enqueue(int data){
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=data; newNode->next=NULL;
    if(!rear){ front=rear=newNode; return; }
    rear->next=newNode; rear=newNode;
}
int front_value(){ return front ? front->data : -1; }
int is_empty(){ return front==NULL; }

int main(){
    enqueue(10); enqueue(20);
    printf("Front: %d\n", front_value());
    return 0;
}
```

```
Sample Output:
Front: 10
```

7. Doubly Linked List

```
#include <stdio.h>
#include <stdlib.h>
struct Node{ int data; struct Node *prev,*next; };
struct Node* head=NULL;

void insert_at_beginning(int data){
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=data; newNode->prev=NULL; newNode->next=head;
    if(head) head->prev=newNode; head=newNode;
}
void insert_at_end(int data){
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=data; newNode->next=NULL;
    if(!head){ newNode->prev=NULL; head=newNode; return; }
    struct Node* temp=head;
    while(temp->next) temp=temp->next;
    temp->next=newNode; newNode->prev=temp;
}
void traverse_forward(){
    struct Node* temp=head;
    while(temp){ printf("%d <-> ",temp->data); temp=temp->next; }
    printf("NULL\n");
}
int main(){
    insert_at_beginning(10);
    insert_at_end(20);
    insert_at_end(30);
    traverse_forward();
    return 0;
}
```

```
Sample Output:
10 <-> 20 <-> 30 <-> NULL
```

8. Circular Linked List

```
#include <stdio.h>
#include <stdlib.h>
struct Node{ int data; struct Node* next; };
struct Node* head=NULL;

void insert_at_beginning(int data){
    struct Node* newNode=(struct Node*)malloc(sizeof(struct Node));
    newNode->data=data;
    if(!head){ head=newNode; newNode->next=head; return; }
    struct Node* temp=head;
    while(temp->next!=head) temp=temp->next;
    newNode->next=head; temp->next=newNode; head=newNode;
}
void traverse(){
    if(!head) return;
    struct Node* temp=head;
    do{ printf("%d -> ",temp->data); temp=temp->next; }while(temp!=head);
    printf("(head)\n");
}
int main(){
    insert_at_beginning(10);
    insert_at_beginning(20);
    traverse();
    return 0;
}
```

```
Sample Output:
20 -> 10 -> (head)
```

9. Binary Tree with Preorder Traversal

```
#include <stdio.h>
#include <stdlib.h>
struct Node{ int data; struct Node *left,*right; };
struct Node* create(int data){
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
    n->data=data; n->left=n->right=NULL; return n;
}
struct Node* insert(struct Node* root,int data){
    if(!root) return create(data);
    if(data<root->data) root->left=insert(root->left,data);
    else root->right=insert(root->right,data);
    return root;
}
void preorder(struct Node* root){
    if(root){ printf("%d ",root->data);
        preorder(root->left); preorder(root->right); }
}
int main(){
    struct Node* root=NULL;
    root=insert(root,50);
    insert(root,30); insert(root,70);
    printf("Preorder: "); preorder(root);
    return 0;
}
```

```
Sample Output:
Preorder: 50 30 70
```

10. Lowest Common Ancestor

```
#include <stdio.h>
#include <stdlib.h>
struct Node{ int data; struct Node *left,*right; };
struct Node* create(int data){
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
    n->data=data; n->left=n->right=NULL; return n;
}
struct Node* find_lca(struct Node* root,int n1,int n2){
    if(!root) return NULL;
    if(root->data==n1 || root->data==n2) return root;
    struct Node* left=find_lca(root->left,n1,n2);
    struct Node* right=find_lca(root->right,n1,n2);
    if(left && right) return root;
    return left?left:right;
}
int main(){
    struct Node* root=create(1);
    root->left=create(2); root->right=create(3);
    root->left->left=create(4); root->left->right=create(5);
    struct Node* lca=find_lca(root,4,5);
    printf("LCA: %d", lca->data);
    return 0;
}
```

```
Sample Output:
LCA: 2
```

11. Find Grandchildren of a Node

```
#include <stdio.h>
#include <stdlib.h>
struct Node{ int data; struct Node *left,*right; };
struct Node* create(int data){
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
    n->data=data; n->left=n->right=NULL; return n;
}
void print_grandchildren(struct Node* root,int value){
    if(!root) return;
    if(root->data==value){
        if(root->left){
            if(root->left->left) printf("%d ",root->left->left->data);
            if(root->left->right) printf("%d ",root->left->right->data);
        }
        if(root->right){
            if(root->right->left) printf("%d ",root->right->left->data);
            if(root->right->right) printf("%d ",root->right->right->data);
        }
        return;
    }
    print_grandchildren(root->left,value);
    print_grandchildren(root->right,value);
}
int main(){
    struct Node* root=create(1);
    root->left=create(2); root->right=create(3);
    root->left->left=create(4); root->left->right=create(5);
    printf("Grandchildren of 1: ");
    print_grandchildren(root,1);
    return 0;
}
```

```
Sample Output:
Grandchildren of 1: 4 5
```

12. Graph Using Adjacency List

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10
struct Node{ int v; struct Node* next; };
struct Node* adj[MAX]={NULL};
struct Node* create(int v){
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
    n->v=v; n->next=NULL; return n;
}
void add_edge(int v,int w){
    struct Node* n=create(w); n->next=adj[v]; adj[v]=n;
    n=create(v); n->next=adj[w]; adj[w]=n;
}
int main(){
    add_edge(0,1); add_edge(0,2);
    printf("Edges added successfully");
    return 0;
}
```

```
Sample Output:
Edges added successfully
```

13. Depth First Search (DFS)

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10
struct Node{ int v; struct Node* next; };
struct Node* adj[MAX]={NULL};
int visited[MAX]={0};
struct Node* create(int v){
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
    n->v=v; n->next=NULL; return n;
}
void add_edge(int v,int w){
    struct Node* n=create(w); n->next=adj[v]; adj[v]=n;
    n=create(v); n->next=adj[w]; adj[w]=n;
}
void dfs(int v){
    visited[v]=1; printf("%d ",v);
    struct Node* temp=adj[v];
    while(temp){
        if(!visited[temp->v]) dfs(temp->v);
        temp=temp->next;
    }
}
int main(){
    add_edge(0,1); add_edge(0,2); add_edge(1,3);
    printf("DFS: "); dfs(0);
    return 0;
}
```

```
Sample Output:
DFS: 0 2 1 3
```

14. Cycle Detection in Undirected Graph

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10
struct Node{ int v; struct Node* next; };
struct Node* adj[MAX]={NULL};
int visited[MAX]={0};
struct Node* create(int v){
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
    n->v=v; n->next=NULL; return n;
}
void add_edge(int v,int w){
    struct Node* n=create(w); n->next=adj[v]; adj[v]=n;
    n=create(v); n->next=adj[w]; adj[w]=n;
}
int detect_cycle_util(int v,int parent){
    visited[v]=1;
    struct Node* temp=adj[v];
    while(temp){
        if(!visited[temp->v]){
            if(detect_cycle_util(temp->v,v)) return 1;
        } else if(temp->v!=parent) return 1;
        temp=temp->next;
    }
    return 0;
}
int detect_cycle(int V){
    for(int i=0;i<V;i++)
        if(!visited[i] && detect_cycle_util(i,-1)) return 1;
    return 0;
}
int main(){
    add_edge(0,1); add_edge(1,2); add_edge(2,0);
    if(detect_cycle(3)) printf("Cycle detected");
    else printf("No cycle");
    return 0;
}
```

```
Sample Output:
Cycle detected
```

15. Social Network - Degrees of Separation (BFS)

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 10
struct Node{ int v; struct Node* next; };
struct Node* adj[MAX]={NULL};
int visited[MAX], distance[MAX];

struct Node* create(int v){
    struct Node* n=(struct Node*)malloc(sizeof(struct Node));
    n->v=v; n->next=NULL; return n;
}
void add_friendship(int u,int v){
    struct Node* n=create(v); n->next=adj[u]; adj[u]=n;
    n=create(u); n->next=adj[v]; adj[v]=n;
}
int degrees_of_separation(int start,int end,int V){
    int queue[MAX], front=0,rear=0;
    for(int i=0;i<V;i++){ visited[i]=0; distance[i]=-1; }
    visited[start]=1; distance[start]=0; queue[rear++]=start;
    while(front<rear){
        int v=queue[front++];
        struct Node* temp=adj[v];
        while(temp){
            if(!visited[temp->v]){
                visited[temp->v]=1;
                distance[temp->v]=distance[v]+1;
                queue[rear++]=temp->v;
                if(temp->v==end) return distance[temp->v];
            }
            temp=temp->next;
        }
    }
    return -1;
}
int main(){
    add_friendship(0,1); add_friendship(1,2); add_friendship(2,3);
    printf("Degrees of separation between 0 and 3: %d",
           degrees_of_separation(0,3,4));
    return 0;
}
```

```
Sample Output:
Degrees of separation between 0 and 3: 3
```